

项目报告

吴品初，张芷瑜，刘礼贤

2025 年 1 月 12 日

1 引言

本项目为 SI100B 课程的最终项目，项目文件已上传到。

我们制作了一款RPG类型类似The Legend of Zelda的游戏。玩家在游戏中扮演一位失忆的女主人公 Ashe，通过与怪物战斗、解谜和探索，逐步揭开隐藏在游戏世界中的关于时间与循环的秘密。玩家可以通过战斗，对话等方式收集残损的齿轮兑换不一样的技能，血量和关键线索剧情（或许可以揭开真正的逃离路线？）

战斗采用弹幕战斗（Bullet Hell），玩家与怪物都可以发射弹幕。

游戏中有三种怪物：飞书、ghost、画像怨灵，shadow（最终boss），每种怪物的属性都不一样，玩家也应该有相应的战斗策略。

玩家击败怪物后可以获得残损的齿轮，同时在探索地图的过程中你会在尘封的柜子或者夹缝中发现残损的齿轮，这些齿轮作为金币，你可以去和商人兑换物品技能，推进剧情。

你需要时刻管理好自己的齿轮与血量。

击败 Boss shadow就能游戏通关，让我们寻找真相，脱离黑暗，逃出生天！

2 项目实施

2.1 场景

2.1.1 场景数量

我们的游戏中一共有四个场景：

1. 教堂钟楼：玩家的初始出生地，场景中散布着齿轮，玩家需要收集这些齿轮以解锁通往下一个场景的路径。
2. 图书馆：玩家需要寻找三本特定的书籍来解锁暗门，场景中充满了各种书籍和书架，提供线索和挑战。
3. 修道院走廊：玩家需要与画像互动，解答谜题，或者尽可能地灵活地躲开画像的追逐，通过这两种方式以进入下一个场景。
4. 教堂的礼拜堂：最终场景，玩家需要不断地和小怪ghost进行战斗收集到足够多的“残损的齿轮”（金币），兑换一些高级技能，与最终 Boss 进行战斗。

我们的场景由地图底层与障碍物构成，在每一帧中，程序会整个地渲染地图渲染出个完整的场景。所有场景均由 Scene.py 中定义的类实现。

2.1.2 大型场景

我们游戏中的每个场景都是中型场景，都有着可以跟随玩家移动而进行视角移动的摄像头。



图 1: 时钟塔楼

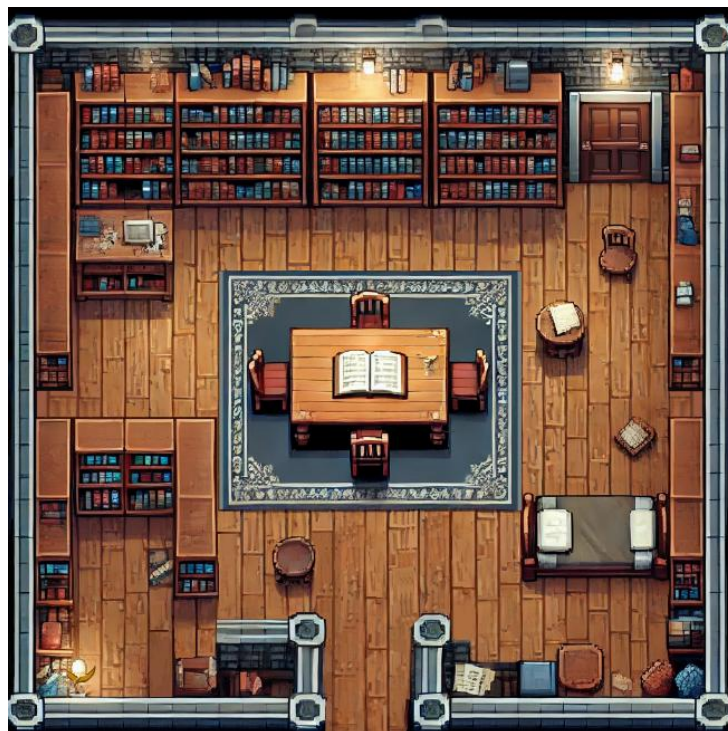


图 2: 尘封的图书馆



图 3: 画廊

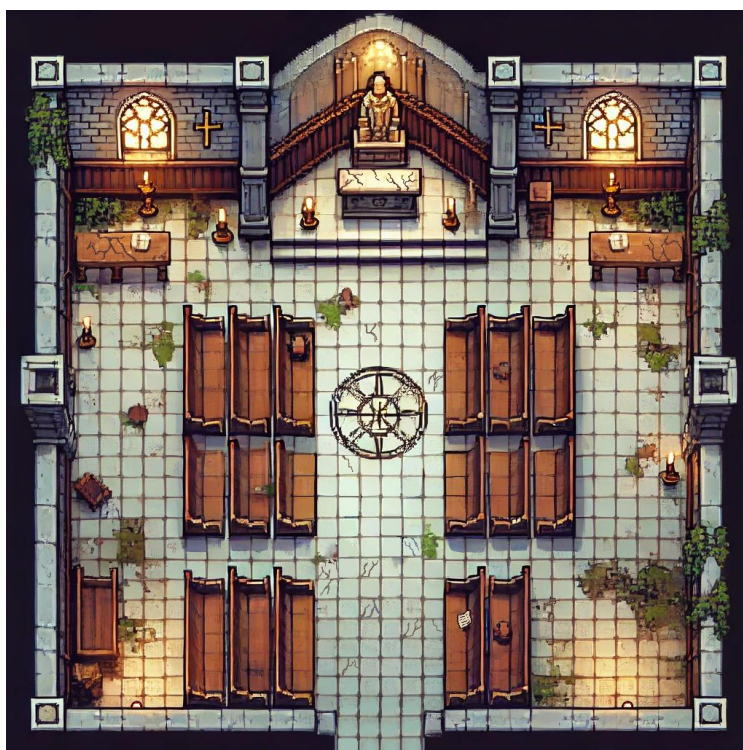


图4: 教堂

镜头移动的原理：每当满足镜头移动的条件时，所有物体（含地砖、NPC、玩家）都会获得一个偏移量，该偏移量等于在这一帧玩家试图移动的距离，在渲染时所有物体的位置都会加上这个偏移量的相反数。因此，玩家移动了 $(dx, dy) + (-dx, -dy) = (0, 0)$ ，即在原地不动，其它物体全部移动偏移量的相反数，达到了镜头移动的效果。

2.1.3 互动物品

我们的游戏中有四种互动物品，分别为椅子、书架、墙壁与残损的齿轮，如图4。

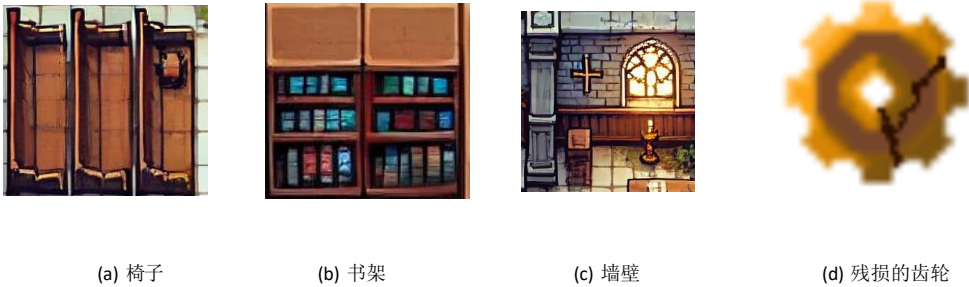


图 4: 互动物品

其中，椅子、书架与墙壁均起到障碍物的作用，玩家无法跨越这些障碍物。而残损的齿轮无法阻挡玩家走动，残损的齿轮随机分布在不同场景的角落，玩家在残损的齿轮旁边可以捡拾它们并且作为交易物使用。这写互动物品会在地图底层上后被渲染，形成它们坐落于地图上的效果。它们起到的作用的实现方法会稍后在游戏机制中提及。

2.2 角色

2.2.1 主要角色

在我们的游戏中，玩家可以通过 `wasd` 四个按键操控主要角色四处移动。主要角色的移动通过检测按键按下与抬起这两种事件的方法来实现，当按键被按下，角色就会持续朝指定方向移动，按键抬起后，角色不再朝指定方向移动。

2.2.2 NPC

友好npc

在场景中存在两种友好 NPC，一种是对话 NPC故事齿轮，还有一种是交易 NPC败北飞书，分别于场景一塔楼的新手教程和击败场景二的飞书亡灵后出现，见图5。

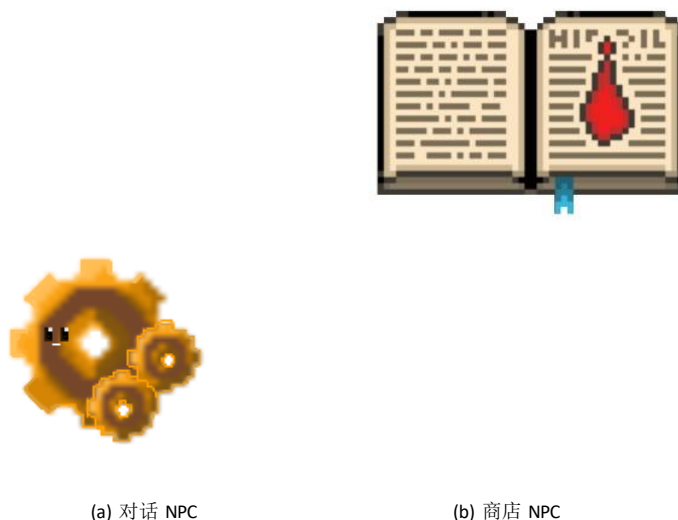


图 5: 友好 NPC

当角色与对话 NPC 碰撞时，按Z会弹出对话框，并且以对话形式告知玩家相关所有按键的绑定，在新手教程结束之后，玩家可以选择按 X 键退出对话；而角色与交易 NPC 碰撞时，按Z会弹出对话框，通过与之对话交易即可实现商品的购买，包括技能的升级，血量的购买等。

简单敌人

在图书馆，三个不同场景中分别存在着三种简单敌人：飞书、ghost、画像怨灵。

1.敌人：飞书（FlyingBook）

定义FlyingBook类，随机飞行并在接触玩家后对玩家进行攻击，对其进行持续性扣血，玩家需通过快速移动来躲避。

战斗互动：飞书飞行持续6秒后掉落，变成合上的状态，玩家需及时走到它身边按“E”键阅读以消灭飞书。

2.敌人：monster（Phantom）

定义monster类，幽灵可实时追踪玩家位置移动并且发射网弹幕（一次三个球），玩家需使用特定攻击方式才能重创幽灵，在攻击过程中别忘了实时移动避免幽灵的攻击伤害你。

战斗互动：使用普通攻击，发射白色弹幕命中幽灵即减血量10；使用火焰，命中幽灵即减血量20；使用圣水，命中幽灵即减血量30。





(a) 飞书

(b) ghost

图 6: 简单敌人

2.2.3 特殊敌人

在最终场景教堂中存在着一一种特殊敌人：shadow，见图7。击败它就会触发游戏结束事件。



图 7:shadow

在我们的程序中，Boss shadow虽然与简单敌人属于两个不同的类，但它们本质上并无不同

Zongde来说属于DarkPriest类，需综合运用之前的所有技能和道具来战斗。

其中微小的区别在于：特殊敌人的属性值很高，弹幕更多、伤害更高，玩家与它的战斗结束后就会迎来结束界面。

2.3 游戏机制

2.3.1 核心机制

游戏的核心机制包括探索、解谜和弹幕战斗。玩家通过操作主要角色移动来与要互动的npc或者物品碰撞，碰撞发生后按下相应按键即会触发相应事件。弹出对话框或者过场动画，战斗采用弹幕制，玩家与怪物通过发射球状的弹幕发起攻击。玩家需要反复击败怪物或者探索地图寻找遗留物以获取“残损的齿轮”（金钱）对自己进行升级和回血，而怪物也会有不同的攻击手段，玩家的最终目标是变得足够强大去挑战 Boss SHADOW。

2.3.2 碰撞系统

主游戏中的碰撞检测写于player.py中，用于检测主地图中与障碍物和npc的碰撞；战斗系统中的碰撞检测写于collision2.py中，用于判断战斗中的像素级碰撞。

2.3.3 资源系统

玩家击败怪物就可获得破碎的齿轮（作为金币），同时在探索地图的过程中也会，玩家获得的金币就越多。金币可以用来在商店中购买升级，提高玩家的战斗力。玩家的血量也是一种资源，玩家需要时刻分配好自己的血量，确保自己不被怪物击杀的同时对地图上的小怪发起攻击，交易系统中也可用金币购买血量。

玩家的资源是玩家类的实例变量。

2.4 游戏性

2.4.1 菜单

我们的游戏有三种菜单，主菜单（界面）、死亡界面与胜利界面。

进入游戏与游戏重新开始时显示主菜单，此时玩家需要按下 ENTER 键以进入游戏。

当玩家死亡时，会出现失败界面，按下 ENTER 键可重新开始这一场景游戏。（？还是直接重新打这个怪？）

当玩家击败Boss时，会出现胜利界面，自动结束游戏，重新打开游戏按下 ENTER 键可重新开始游戏。

2.4.2 游戏按键

游戏中玩家通过键盘可以实现人物多种动作和行为。

在主游戏界面中，我们绑定了按键wasd作为控制人物上下左右的方向键，而对话则是使用z继续，x取消并且退出对话，开门解锁并且切换下一个场景则是使用N键。

在战斗界面中，我们则使用space空格键发射弹幕进行攻击，通过鼠标指针控制发射弹幕的方向。角色的移动和攻击动作通过检测按键事件实现，确保响应迅速且准确。

2.5 代码

仓库中 Game Project 为游戏文件。用编辑器打开 Game Project 后运行 main.py即可进入游戏。程序采用面向对象编程的编程方式，代码中难以理解的地方全都添加了注释。

下面罗列程序中的各个文件的相应作用：

一、文件名中无“2”的部分：

1、block.py 该文件定义了 block 类，这是游戏中的障碍物。

- 2、dialog.py该文件定义了DialogBox类，这是游戏中的对话框部分。
- 3、main.py该文件是游戏主程序，包含了游戏主循环，负责运行游戏。
- 4、Map.py该文件定义了build_map和load_map函数，负责加载出地图。
- 5、npc.py该文件定义了npc类，负责管理游戏中的所有npc。
- 6、player.py该文件处理了主游戏场景中玩家的碰撞，以及定义了Player类，负责管理玩家的操作。
- 7、sence.py该文件定义了scene_control类，负责主游戏场景加载。
- 8、settings.py该文件负责管理主游戏中的各种参数。

二、文件名有“2”的部分：

- 1、Boss2.py 该文件定义了boss类，这是战斗模块中的boss。
- 2、Bullets2.py该文件定义了Bullet_M和Bullet_P类，是游戏的战斗场景中玩家与怪物发射出的弹幕。
- 3、collision2.py该文件定义了check_collision类，便于判断战斗中的像素级碰撞。
- 4、Flying_book2.py该文件定义了Flying_book类，这是战斗模块中的飞书怪。
- 5、Main2.py该文件是战斗主程序，在接收main.py的指令后开始运行，用于运行战斗。
- 6、Map2.py该文件用于加载战斗场景的地图。
- 7、Monster2.py该文件定义了Monster类，这是战斗模块中的小怪。
- 8、Player2.py该文件定义了Player类，这是战斗模块中的玩家。
- 9、SceneManager2.py该文件负责加载战斗中的场景。
- 10、Settings2.py该文件负责管理战斗中的各种参数。
- 11、shop2.py该文件定义了shop类，负责处理商店的显示与操作。

3 创意

3.1 内置谜题

追逐战和谜题

在游戏的第三个场景——破旧的修道院长廊中，我们巧妙地融入了追逐战与谜题元素。玩家操控着Ashe的人物模型，在昏暗且悠长的走廊上紧张地奔跑，躲避画像怨灵的紧追不舍。一旦被画像怨灵追上，玩家将被强制带入交互界面，此时必须回答由openai大模型随机生成的Puzzle类谜题。若答错两题，Ashe便会遭遇不幸，游戏结束。

其中由ai自动检测玩家回答问题的正确与否，接着根据相应回答给出“YOU ARE WRONG!”或者“YOU ARE RIGHT!”的大方向答案和ai随机生成的情景对话。

例如：

“In twilight’s hush, where shadows play,
A reflection stares back on the way.
The answer lies in moonlight’s gentle beam,
Yet, it’s not found until morning’s dark dream.”

当然如果你在操纵过程中足够灵活，或许你可以一个谜题也不用回答无伤不费脑子的通关。这是给游戏高手设置的额外奖励！

3.2 碰撞系统

游戏采用像素级碰撞检测，确保玩家与怪物、障碍物等的交互更加精确和真实。像素级碰撞检测通过逐像素检查碰撞区域，提供了更高的精度和更自然的碰撞效果。这种碰撞检测机制不仅提高了游戏的公平性和挑战性，还为玩家带来了更加沉浸的游戏体验。

玩家的攻击可以精确到单个像素，使得攻击更加精准。例如，玩家可以使用特定的技能来攻击怪物的弱点，这些弱点只有几个像素大小。在攻击和技能释放时，可以添加像素级的粒子效果。这些粒子可以精确地从攻击点发射，形成独特的视觉效果，增强战斗的视觉冲击力。

见图9。

图 9: 战斗界面

3.3 战斗界面

游戏的攻击界面采用独立于游戏本体的窗口的形式，当在主游戏界面中碰见怪物npc就会触发相应对话或者直接跳转到战斗界面。这种独立式的战斗界面使战斗过程更流畅丝滑，并且颇有模仿“斗兽场”的感觉，使得玩家可以沉浸式体验剧情的同时进行有趣的战斗。

3.4 攻击方式/战斗机制

在游戏的战斗场景中，玩家需要操控Ashe的人物二维模型灵活移动以躲避怪物的攻击。不同怪物拥有各具特色的攻击方式，这不仅增加了游戏的趣味性，还诱导玩家积极探索游戏机制，寻找最适合的通关策略。例如，在图书馆关卡中，我们创新性地设置了“飞书”类怪物。飞书在空中飞行3秒后便会掉落，变成合上的状态，玩家需迅速走到飞书身边，按下“E”键阅读，以此来消灭飞书。这种非进攻类的交互方式，与后续飞书转变为友好NPC与玩家进行交易的情节形成了巧妙的呼应。而在与“ghost”的战斗中，需要同时进行躲避ghost攻击，定位魔法弹发射位置，发射魔法弹的三重行为，多按键的同时运用也成为了游戏战斗的难点所在。